



How does an admissible heuristic ensure an optimal solution?

Asked 9 years, 6 months ago Modified 2 years, 2 months ago Viewed 40k times



22



When using A* (or any other best path finding algorithm), we say that the heuristic used should be **admissible**, that is, it should never overestimate the actual solution path's length (or moves).

How does an admissible heuristic ensure an optimal solution? I am preferably looking for an intuitive explanation.

If you want you can explain using the **Manhattan** distance heuristic of the **8-puzzle**.

algorithms artificial-intelligence search-algorithms heuristics

Share Cite Improve this question

Follow

edited Feb 24, 2019 at 14:59

user20691

asked Oct 14, 2013 at 6:34



Ashwin

261

1

2

10

- 2 @Ashwin Intuitively because when the algorithm finds a path of length k , it has already tried *every other path which can possibly be of length at most k* . That's why your heuristic function must *never overestimate* the cost to the goal. Try it yourself by making a heuristic function that might overestimate. – Pål GD Oct 14, 2013 at 11:24

3 Answers

Sorted by:

Highest score (default)



10



While Anton's answer is absolutely perfect let me try to provide an alternative answer: being admissible means that the heuristic does not overestimate the effort to reach the goal, i.e., $h(n) \leq h^*(n)$ for all n in the state space (in the 8-puzzle, this means just for any permutation of the tiles and the goal you are currently considering) where $h^*(n)$ is the optimal cost to reach the target.

I think the most logical answer to see why A^* provides optimal solutions if $h(n)$ is admissible is because it sorts all nodes in OPEN in ascending order of $f(n) = g(n) + h(n)$ and, also, because it does not stop when generating the goal but when expanding it:

1. Since nodes are expanded in ascending order of $f(n)$ you know that no other node is more promising than the current one. Remember: $h(n)$ is admissible so that having the lowest

$f(n)$ means that it has an opportunity to reach the goal through a cheaper path that the other nodes in OPEN have not. And this is true unless you can prove the opposite, i.e., by expanding the current node.

2. Since A^* stops only when it proceeds to expand the goal node (as opposed to stop when generating it) you are sure (from the first point above) that no other node leads through a cheaper path to it.

And this is, essentially, all you will find in the original proof by Nilsson et al.

Hope this helps,

Share Cite Improve this answer Follow

answered Oct 14, 2013 at 22:14



Carlos Linares López

3,433 14 30

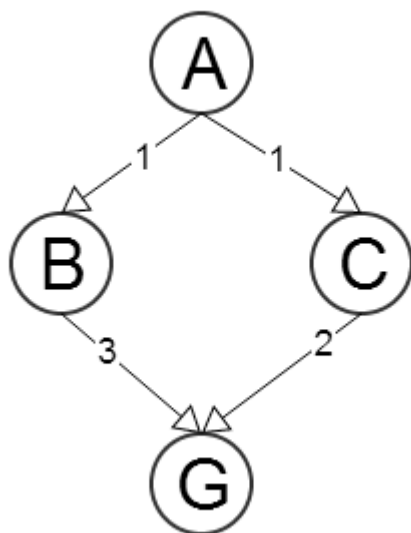
- 4 Thanks. It helped. You were referring to some proof by Nilsson et al. Who is that? And where can I find the proof? – Ashwin Nov 9, 2013 at 11:19



20



Look at this simple example.



Let A and G be respectively the *starting* and *goal* nodes. Let $h(N)$ be an estimate of the path's length from node N to G , $\forall N$ in the graph. Moreover, let $c(N, X_i)$ be the *step cost function*

from node N to its neighbour X_i , $\forall N$ and $i = 1..m$, where m is the number of neighbours of N (i.e., a function that returns the cost of the edge between node N and one of its neighbours).

Let the heuristics be

- $h(B) = 3$
- $h(C) = 4$

This heuristics function H is not admissible, because

$$h(C) = 4 > c(C, G) = 2$$

If the A^* algorithm starts initially from node A , it will select next node B for expansion and, after this, it will reach node G from there. And the path will be $A \rightarrow B \rightarrow G$ with cost 4, instead of $A \rightarrow C \rightarrow G$ with cost 3. If the heuristic function was admissible this would not have happened.

Share Cite Improve this answer

edited Jan 4, 2017 at 15:59

answered Oct 14, 2013 at 11:06

Follow

user20691



Anton

954 6 14

1 Ok. But how does an admissible heuristic ensure an optimal solution? – Ashwin Oct 14, 2013 at 11:21

It can happen that - $h(b) < h(c)$ with both $h(b)$ and $h(c)$ being admissible, but $\text{actual_cost}(b) > \text{actual_cost}(c)$ right? So b will be chosen as the next path where as in reality c would have given the best path. – Ashwin Oct 14, 2013 at 11:25

For the 1st comment: admissible heuristics ensures to find the shortest path. The solution itself is optimal if the heuristic is consistent. – Anton Oct 14, 2013 at 12:05

For the 2nd comment: if the heuristic is admissible $A \rightarrow B$ can be chosen for the next node to expand, but after that the A^* will chose $A \rightarrow C$ and not $A \rightarrow B \rightarrow G$. And at the end it will end up with $A \rightarrow C \rightarrow G$. – Anton Oct 14, 2013 at 12:08

1 Because A^* works like this. It expands the node with least sum of distance to that node + heuristic estimation from that node. $d(A, G) + h(G) = 4 + 0 = 4$ and $d(A, C) + h(C) = 1 + \text{something} \leq 2$ (because it is admissible). So C has lower sum and the A^* will chose it. The same way it will than expand G and find the least path. – Anton Oct 14, 2013 at 19:14

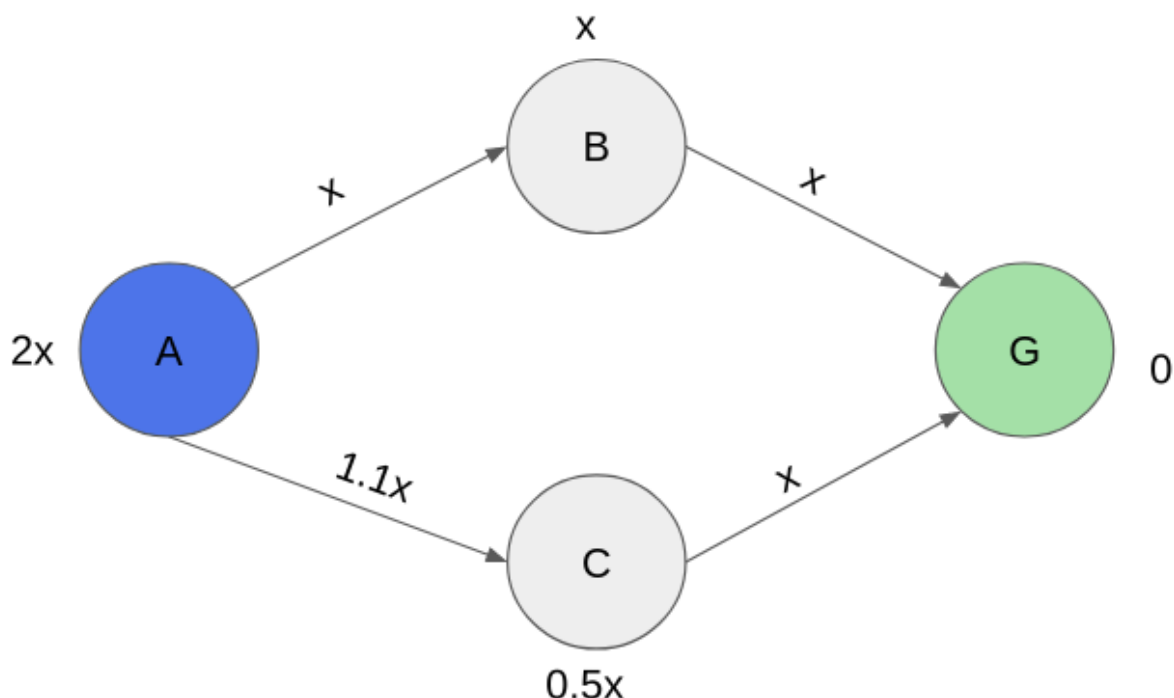


3



I'd like to expand upon Anton's comment in his answer, and provide an explicit answer to the situation posed by Ashwin in the comments. I think it'll be helpful to answering the primary question.

Let's consider the situation



Where A is the starting node and G is the goal node. The numbers on the nodes are the heuristic costs, while the numbers on the edges are the costs to travel between those two nodes.

We can see that the heuristic function is admissible (i.e it doesn't overestimate the cost of reaching the goal) and consistent (ie. it decreases as we get closer to the goal node).

Here's how the A* algorithm would find the optimal solution for this graph:

Iteration 1: It checks the $f(x)$ for all of A's neighbours

- $f(B) = x + x = 2x$
- $f(C) = 1.1x + 0.5x = 1.6x$

Since $f(C)$ is smaller, it picks C as the next node.

Iteration 2: Check $f(x)$ for C's neighbours and the existing paths

- $f(B) = x + x = 2x$ (A->B)
- $f(G) = 2.1x + 0 = 2.1x$ (C->G)

Clearly, the path from A->B is cheaper and thus A* will shift its focus to that path.

Iteration 3: Check $f(x)$ for B's neighbours and the existing paths

- $f(G) = 2x + 0 = 2x$ (B->G)
- $f(G) = 2.1x + 0 = 2.1x$ (C->G)

Since $B \rightarrow G$ is cheaper here and we've reached the goal node (plus traversed all paths), the algorithm ends with the (optimal) solution $A \rightarrow B \rightarrow C$.

[Share](#) [Cite](#) [Improve this answer](#) [Follow](#)

answered Feb 9, 2021 at 20:41



[Aayush Kucheria](#)

31 2